Lane Detection - Using Machine Learning

Project: Deep Learning Based Lane Detection System

1. Problem Context & Proposed Approach

Lane Vision AI is a deep learning based semantic segmentation system that detects lane boundaries from road images and driving video streams.

The system leverages a Fully Convolutional Neural Network (FCN) to perform pixel-wise classification of lane markings, enabling accurate lane detection under real-world driving conditions.

Unlike traditional rule-based computer vision approaches, this solution uses end-to-end deep learning to improve robustness against:

- Curved roads
- Variable lighting conditions
- Worn or partially visible lane markings
- Shadows and environmental noise

System Deliverables

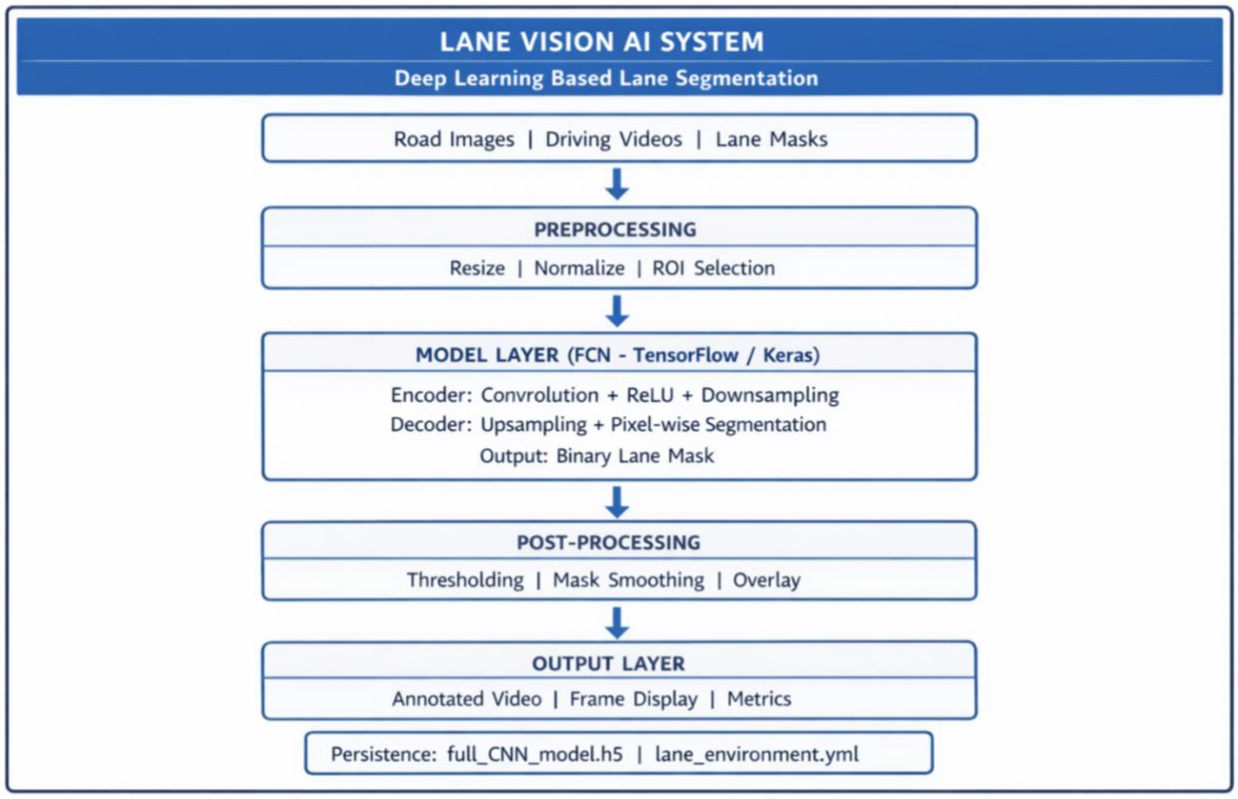
Output	Description
Lane Segmentation Mask	Binary mask identifying lane pixels
Overlay Visualization	Detected lanes projected onto original frame
Annotated Video	Processed driving video with highlighted lanes
Model Performance Metrics	Loss curves and segmentation accuracy

---

Core Objective

To design a scalable and real-time lane detection system suitable for integration into Advanced Driver Assistance Systems (ADAS).

2. Architectural Design Overview



3. Tech Stack Summary

Layer	Tool / Framework	Role
Programming Language	Python 3.8+	Core development language
Deep Learning Framework	TensorFlow + Keras	Build and train FCN model
Model Architecture	Fully Convolutional Network (FCN)	Pixel-wise lane segmentation
Computer Vision	OpenCV	Image processing & frame handling
Data Processing	NumPy	Matrix and tensor manipulation
Visualization	Matplotlib	Training loss and performance plots
Video Processing	MoviePy	Frame-by-frame video transformation
Model Storage	.h5 (Keras Model)	Persist trained neural network
Environment Management	Conda	Dependency & environment control
Development Environment	Jupyter Notebook / Python Scripts	Model training and testing

---

Why This Combination of Tools?

- TensorFlow/Keras enables efficient CNN model construction.
- Fully Convolutional Networks are ideal for semantic segmentation.
- OpenCV provides reliable real-time frame processing.
- MoviePy enables scalable video inference.
- Conda ensures reproducibility across systems.

This stack ensures modularity, scalability, and future deployment capability.

4. Operational Pipeline

Step 1 – Data Conditioning

- Resize and normalize road images
- Convert lane masks to binary format
- Extract frames from input videos

Step 2 – Network Training

- Images passed into Fully Convolutional Network (FCN)
- Encoder extracts road features
- Decoder generates lane segmentation mask
- Loss: Binary Cross Entropy
- Optimizer: Adam

Step 3 – Deployment Inference Flow

Video Input
↓
Frame Extraction
↓
FCN Model Prediction
↓
Binary Lane Mask
↓
Overlay on Original Frame
↓
Annotated Output Video

Step 4 – Performance Assessment

- Pixel accuracy measurement
- Loss curve analysis
- Visual validation of detected lanes